Background: Stellar spectra are the fingerprints of stars

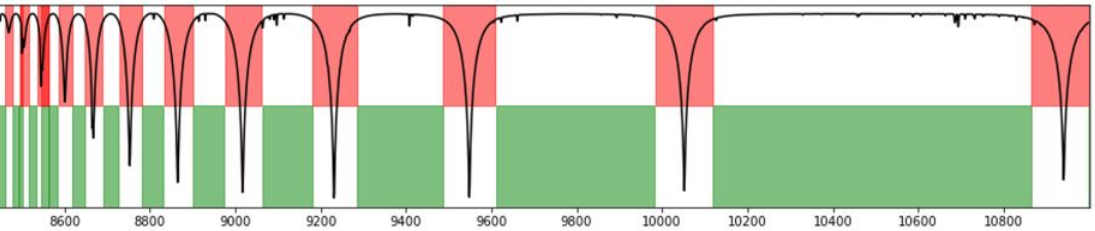
A star's light is spread into a spectrum (wavelength vs flux). Each element in star absorb a set of specific colors, results in absorption lines.

These lines hold the key to a star's composition, temperature, and luminosity.

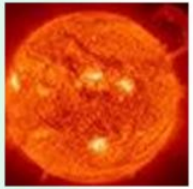
Line features & Properties

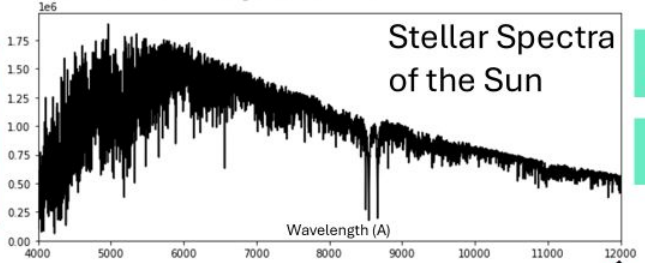
- Sensitive to Positions: Line presence & shifting
- Global & Local dependencies: Line strength, width, and ratios; overall shapes.

Hydrogen absorption lines: comoving and spreading all over.



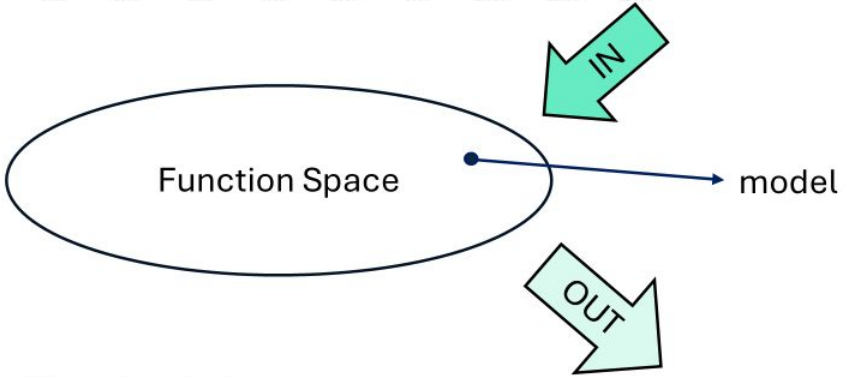
The Problem: Parameter inference from stellar spectra





Wave Grid [3000, ..14000]

Flux Array O(10,000)



Physical Parameters:

- Temperature (Teff)
- Gravity (log g)
- Metallicity ([M/H]) w.r.t Sun
- Carbon (C) w.r.t Sun
- Oxygen (O) w.r.t. Sun

Teff = 5777K

Log g = 4.45

[M/H] = 0.0

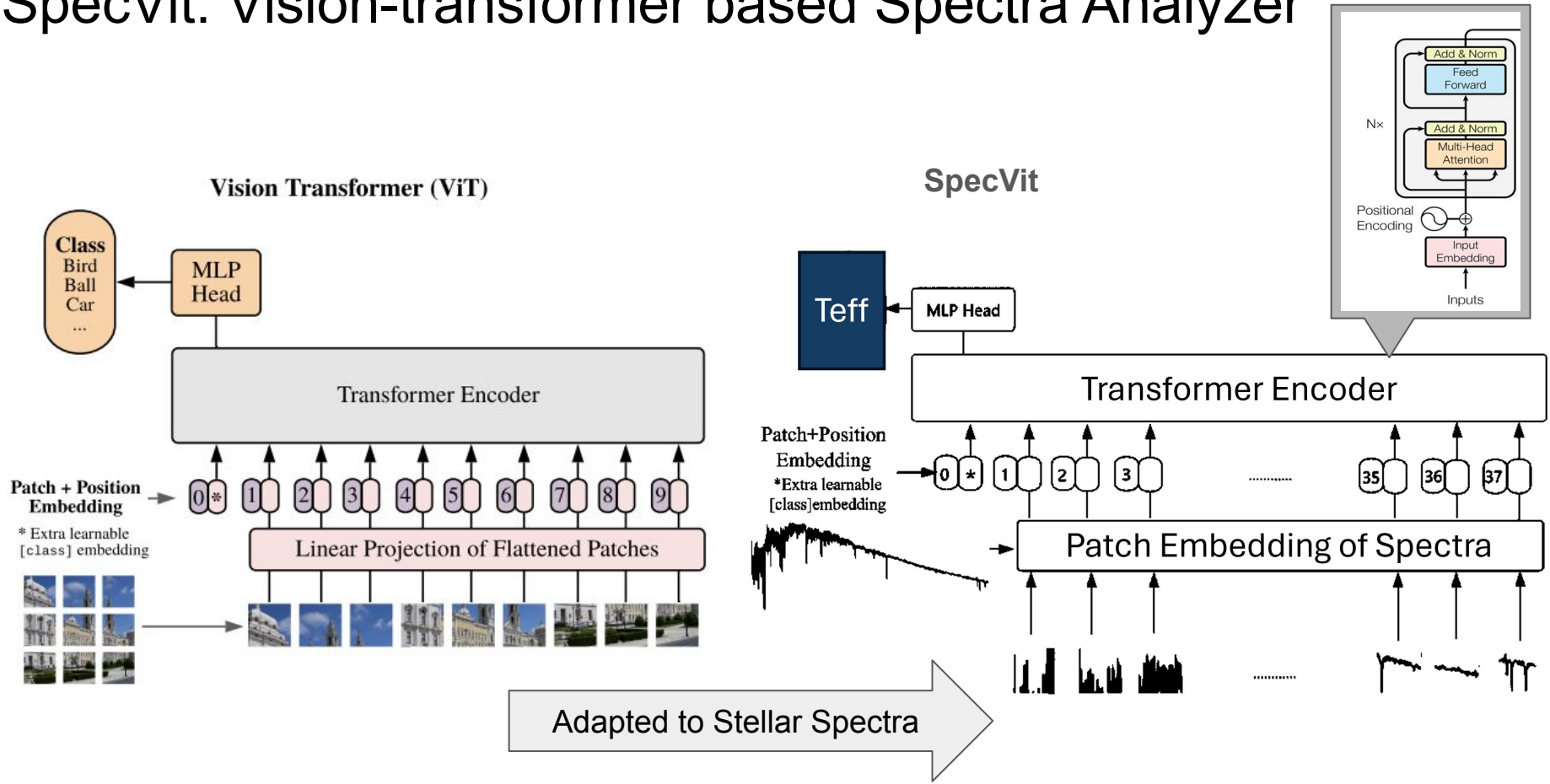
C = 0.0

O = 0.0

Why transformer?

- Attention performs better with less training time
  - Based solely on attention mechanisms (no RNN or CNN), the Transformer performs superior in NLP tasks while being more parallelizable and requiring significantly less time to train. (1706.03762)
- **Long-Range Dependencies (> CNN, RNN)**
  - CNNs assume translational invariance & distortion stability, not applicable to stellar spectra (2306.15703 Yuan-Sen Ting)
  - CNN is a special case of attention (1911.03584)
  - RNN is Transformer with linear attention (2006.16236)
  - While CNN and RNN are limited to identifying locally correlated features through fixed-size filters, attention can discern correlations across the entire spectrum. This global perspective could significantly enhance feature extraction efficiency.
  - Capturing orders with **Positional Encoding** rather than sequential recurrence leads to massive speed-up.
- **Spotlight Focus (> MLP)**
  - Attention acts like a spotlight, allowing the model to prioritize specific elements of the input relevant to the current task. Fully connected layers treat all inputs equally, which might not be ideal for understanding complex relationships.

SpecVit: Vision-transformer based Spectra Analyzer

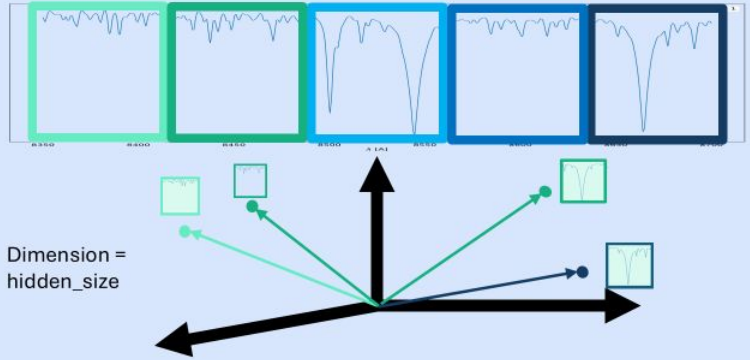


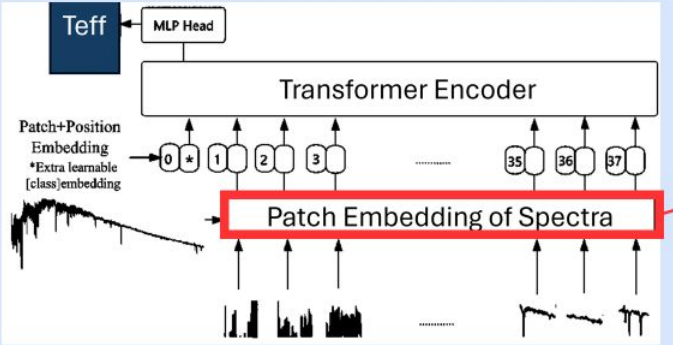
1. Spectra Patch Embedding

Similar to how audio or images are converted into sequences of vectors using a sliding window, we segment spectra into 'words' (sequences of embedded vectors). However, unlike images, precise position information is crucial for spectral analysis.

Goal:

- Robust to noise and subpixel shifts
- Minimize information loss.





C1D: 1D Convolution Based Patch Embedding

Pro: Designed to exploit the inherent spatial relationships within 1D spectra. Effectively capture localized features (i.e identifying specific absorption line shapes, capturing gradual trends within the spectrum)

Con: Assume translational invariance & distortion stability, not applicable to spectra (2306.15703 Yuan-Sen Ting). Loss info.

```
self.projection = nn.Conv1d(self.num_channels, self.hidden_size,
kernel_size=self.patch_size, stride=self.patch_size)
```

SW: Sliding Window Based Patch Embedding

Pros: No information loss.

Con: Ignores pixel order during matrix multiplication, potentially missing localized spectral features.

```
x = x.unfold(dimension=1, size=self.patch_size, step=self.stride_size)
```

Ran 120 experiments with the following parameters:

Stride Ratio : 0.2, 0.4, 0.6, 0.8, 1
Patch Size : 50, 100, 200, 500
PatchEmbed: SW, C1D
Noise_level : 0.1, 1, 2

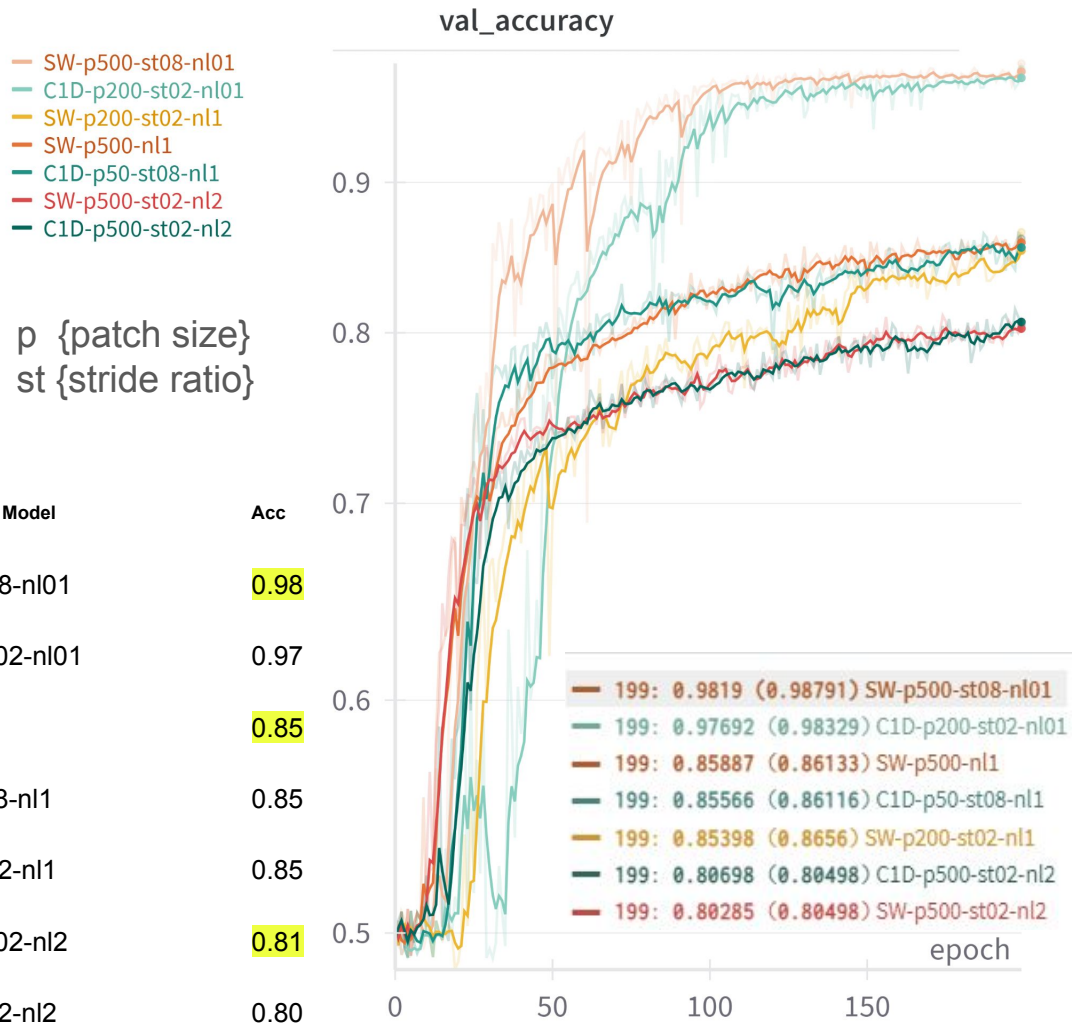
https://wandb.ai/ydz627-google/vit-class-SW-C1D-final-final/table

ViT Results:

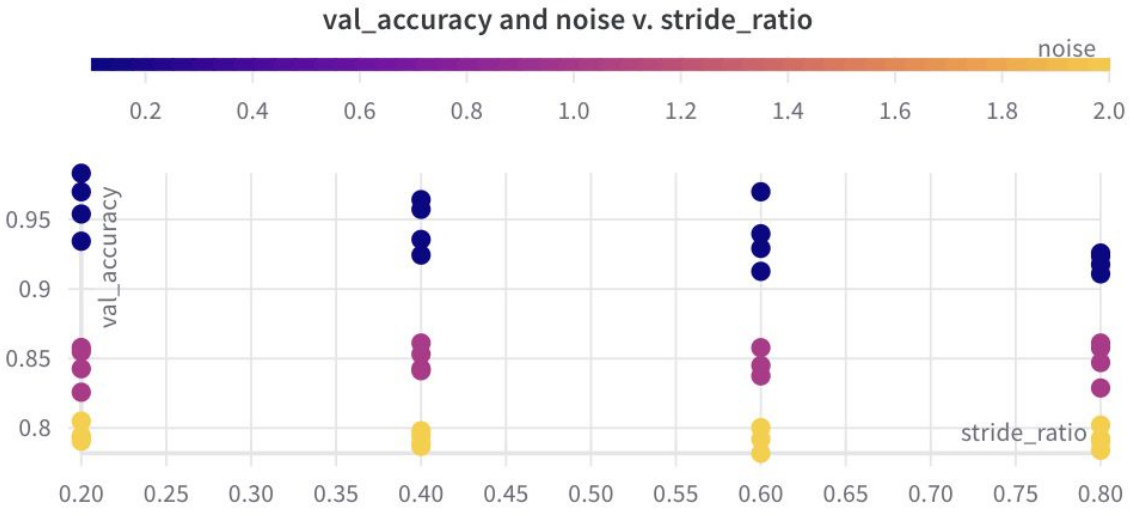
The only significant differentiator is the noise level; Other designs and parameters show minimal impact.

p {patch size}
st {stride ratio}

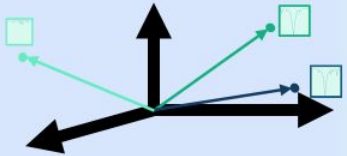
Noise	Model	Acc
0.1	SW-p500-st08-nl01	0.98
	C1D-p200-st02-nl01	0.97
1.0	SW-p500-nl1	0.85
	C1D-p50-st08-nl1	0.85
	SW-p200-st02-nl1	0.85
2.0	C1D-p500-st02-nl2	0.81
	SW-p500-st02-nl2	0.80



Little intuition gained from parameter variations: stride ratio



2. Spectra Embedding



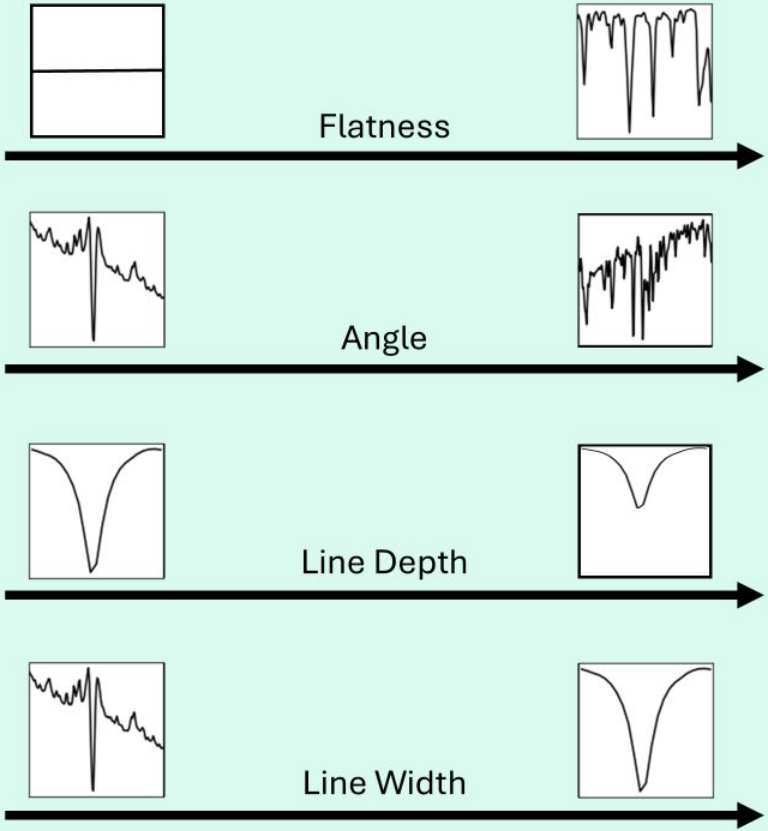
Learnable Positional Embeddings	• Capture long-range dependencies more effectively. • Size: (1, #num patches, hidden) = (1, 154, 84)
Learnable [CLS] token	• Like BERT, the [CLS] token is added to the beginning of the embedded sequence and is used to classify the entire spectra. (1, 1, 84)

The Embeddings class takes in a batch of hidden vectors and (1) prepend a [CLS] token; (2) add a same sized positional embedding directly on top of the input patch embeddings. (3) dropout

- Input Size:
  - (batch_size, num_channels , wave_dim) = (16, 154, 84)
- Output Size:
  - (batch_size, num_patches, hidden_size) = (16, 155, 84)

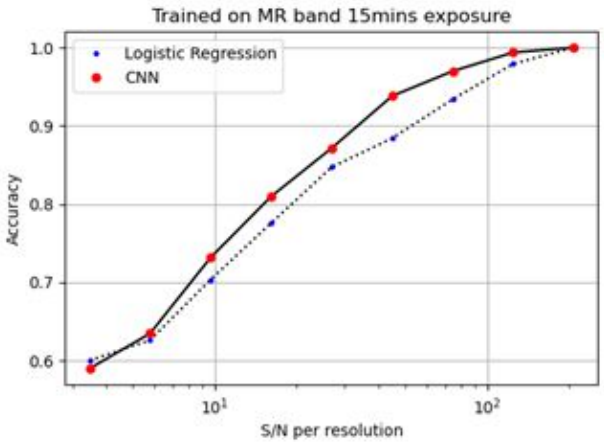
```
self.position_embeddings = nn.Parameter(torch.randn(1,
self.patch_embeddings.num_patches + 1, config["hidden_size"]))
self.cls_token = nn.Parameter(torch.randn(1, 1, config["hidden_size"]))

x = torch.cat((cls_tokens, x), dim=1) # (16, 154, 84) > (16, 155, 84)
x = x + self.position_embeddings
x = self.dropout(x)
```

Expected Spectra Embeddings: Interpretable clustering, meaningful directions similar to word embeddings.
(Queen – female + male = King)



Comparison to traditional ML

Logistic Regression: > 82% CNN: > 85%



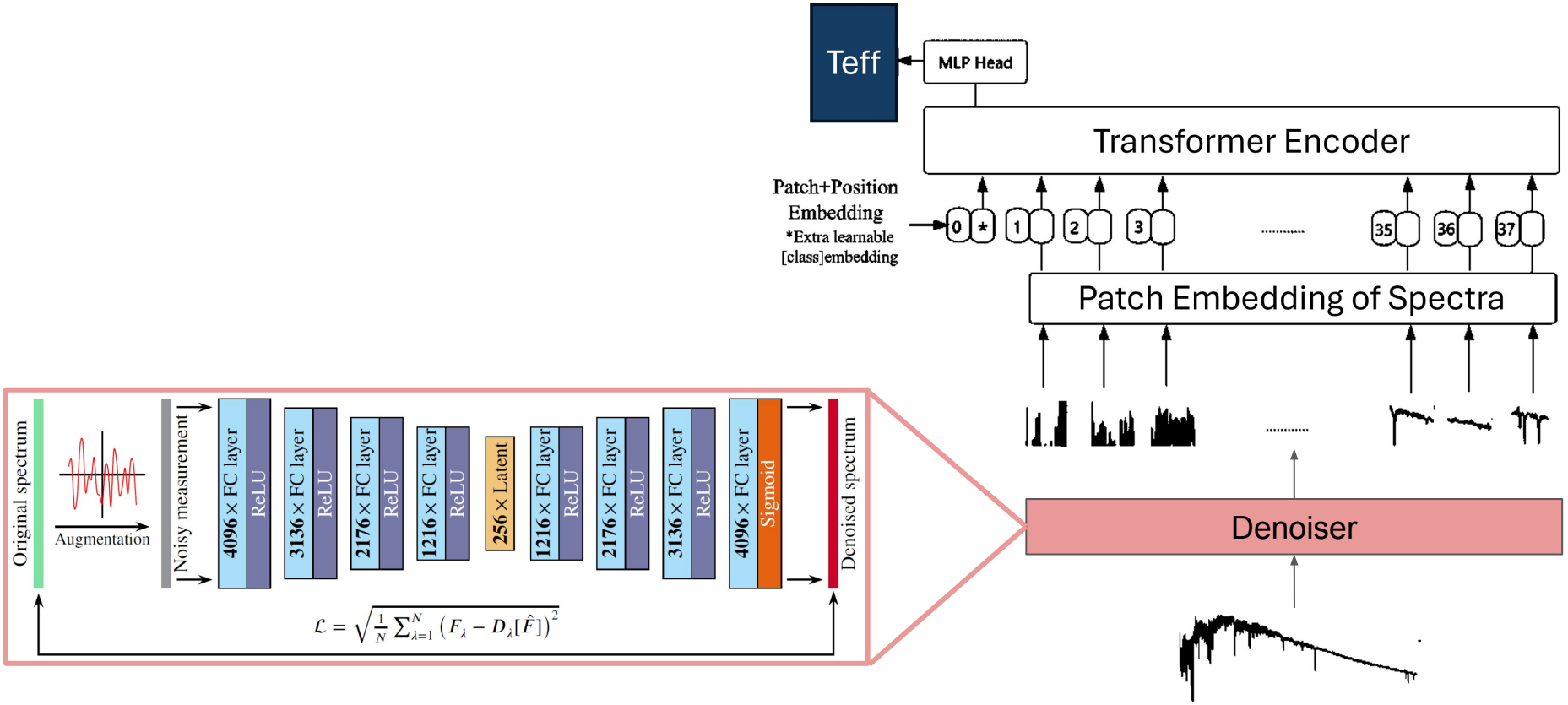
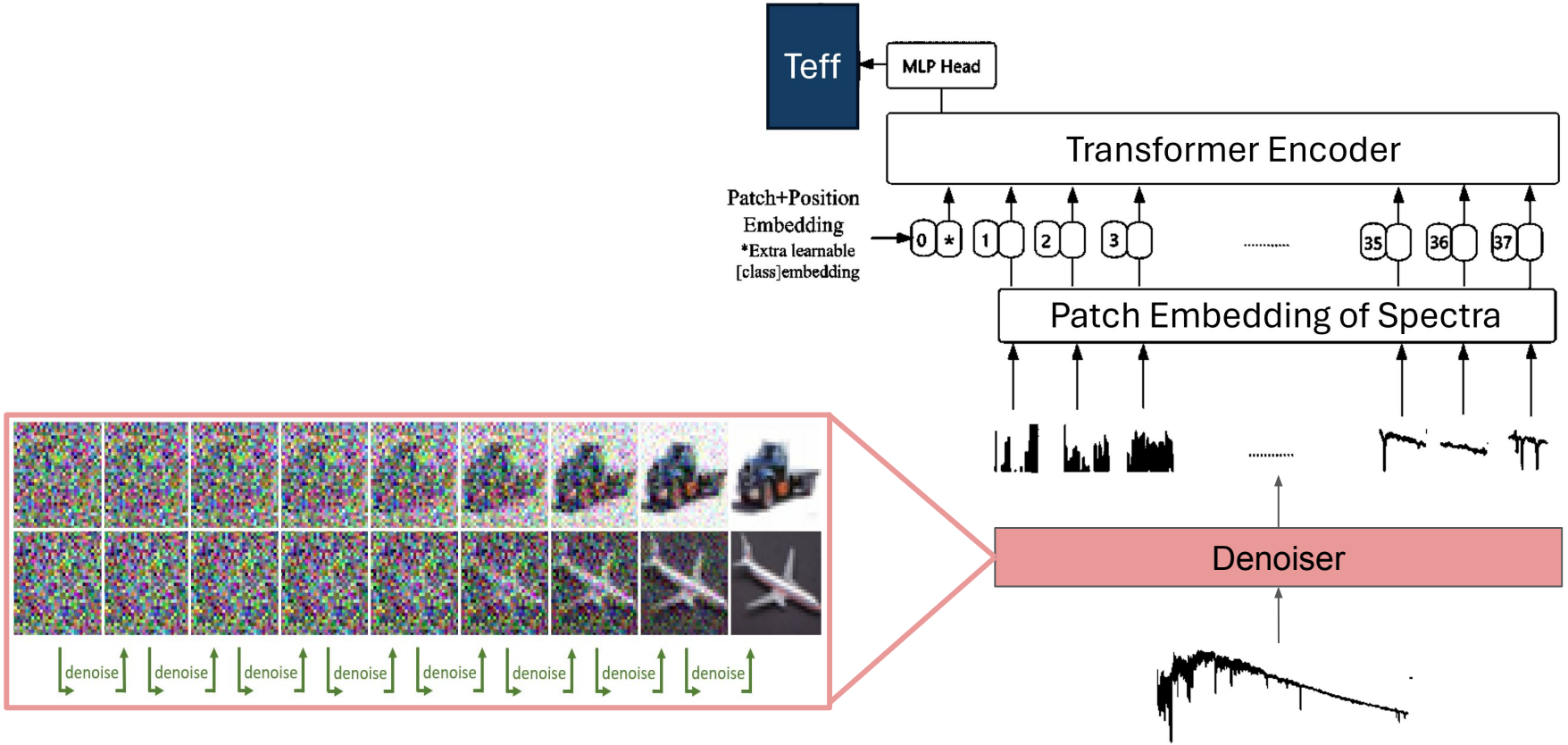
SpecVit:

Noise	Model	Acc
0.1	SW-p500-st08-nl01	0.98
	C1D-p200-st02-nl01	0.97
1.0	SW-p500-nl1	0.85
	C1D-p50-st08-nl1	0.85
	SW-p200-st02-nl1	0.85
2.0	C1D-p500-st02-nl2	0.81
	SW-p500-st02-nl2	0.80

SVM: > 93%

	train_noise	test_noise	score
0	0.0	0.0	0.972993
1	0.1	0.0	0.975994
2	1.0	0.0	0.963991
3	10.0	0.0	0.765941
4	0.0	0.1	0.968492
5	0.1	0.1	0.975244
6	1.0	0.1	0.961740
7	10.0	0.1	0.764441
8	0.0	1.0	0.902476
9	0.1	1.0	0.877719
10	1.0	1.0	0.933233
11	10.0	1.0	0.764441
12	0.0	10.0	0.582896
13	0.1	10.0	0.560390
14	1.0	10.0	0.633908
15	10.0	10.0	0.723181

Denoiser + ViT



Denoiser + ViT